

CIVICCONNET PROJECT : MILESTONE 1

Prepared by Ayanda Mabena, Nompilo Mbense and Refilwe Segele

Contents

Contents	1
Requirements, Traceability, and Scope Lead	3
Problem and Business Need	3
Stakeholder Analysis	4
Scope Baseline	5
Functional Requirements.....	6
Non-functional Requirements	8
Requirements Traceability Matrix	8
CONSTRAINTS AND TRADE-OFFs ANALYSIS	10
Initial Risk Register	11
Engineering Decision Log.....	12
Repository and Team Governance	12
Github governance	13
Team Gorvenance	13
AI Compliance and controls.....	14
Conclusion	15
References	15

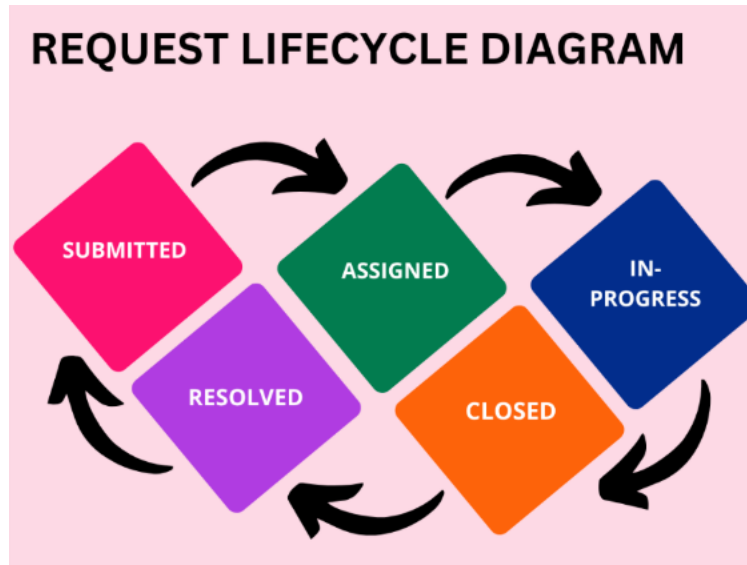
Requirements, Traceability, and Scope Lead

Problem and Business Need

The organisation manages service requests through a mix of communication methods, including phone calls, emails, WhatsApp messages, spreadsheets, and paper records. Because there is no single controlled record of a request's lifecycle, it is difficult to know where a request stands, who is handling it, what has been done, and whether it has been resolved. Complete tracking throughout a request's lifecycle is recognised as a core benefit of structured service request management, ensuring all parties stay informed of progress (ServiceNow, 2026) .

This fragmentation creates real frustration on both sides. Requesters often end up sending duplicate requests or following up across multiple channels just to get an answer, wasting their time. Staff face a similar problem: the same issue may surface across several platforms, forcing them to investigate or resolve it more than once. The result is a slower, more time-consuming process with delayed resolutions all round.

What the organisation needs is a system that allows people to submit and monitor requests, directly supporting the core business need to submit, manage, and monitor service requests in one place (ServiceNow, 2026) . Beyond that, the organisation needs clear ownership of every request. This includes who is responsible for it, what priority it carries, and what actions have been taken. Right now, that need isn't being met because accountability is weak for changes to request status and the actions taken. Solving this problem creates value across the organisation. Requesters gain visibility and clarity on the status of their requests. Staff save time and can delegate tasks more effectively and manage their workload with greater control. Management gains reliable, auditable data that supports accountability and service-performance reporting.

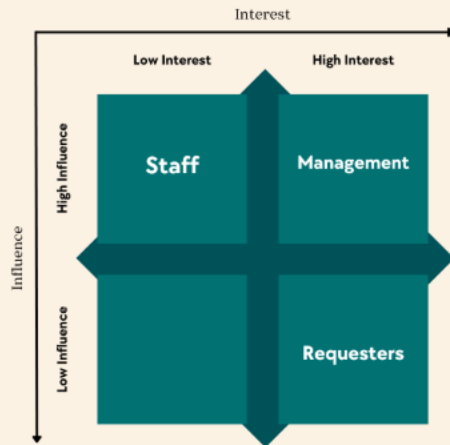


Stakeholder Analysis

Requesters need to be able to submit a new service request, categorise it, and view the status of their submitted requests. Staff need to be able to view requests and search, filter, or sort them, as well as assign or accept responsibility for a request. Management needs to be able to identify open, overdue, resolved, and closed requests.

One conflict identified is between Requesters and Staff. Requesters want the system to be fast and straightforward, allowing them to submit a request quickly and see results soon after. This may clash with Staff's needs, since resolving a request properly takes time, requires the Requester to provide sufficient detail upfront, and depends on staff having the capacity to investigate and act on it. A system optimised purely for speed on the Requester's side risks producing incomplete requests that are harder and slower for Staff to resolve. One functional requirement derived from this is that the system must allow a Requester to view, update, or delete their submitted request where appropriate.

STAKEHOLDER ANALYSIS



Scope Baseline

In-scope

- Requesters can submit a new service request with relevant details.
- Requesters can categorise their request using a controlled category list, for instance, assign it to maintenance or IT support.
- Requesters can view the status of their submitted request.
- Requesters can view a list of their previously submitted requests.
- Requesters receive feedback when their request is accepted or completed.
- Staff can view requests relevant to them and search/filter/sort by useful criteria.
- Staff can view full request details.
- Staff can assign or accept responsibility for a request.
- Staff can update a request status from open to in progress and so forth.
- Staff can record actions, comments, or resolution notes on a request.
- Management can view service activity, including open, overdue, resolved, and closed requests.
- Management can view requests by category for basic reporting and accountability purposes.
- Role-based access so Requesters, Staff, and Management see only what's relevant to their role.

Out-of-scope

- Native mobile app — web-based only.

- Integration with external channels (auto-importing requests from WhatsApp, phone calls, or email into the system).
- Multi-language support.
- Advanced analytics or AI-driven prioritisation/auto-assignment of requests.
- Public API for third-party integration.
- Payment processing or billing functionality.

Deferred scope

- Automated notifications (SMS/emails) on status change.
- Advanced management dashboards/reporting beyond basic status counts,
- Public API for future integrations.

Automated notifications have been deferred, as this milestone prioritizes establishing the core, controlled lifecycle of a request. This includes submission, assignment, status tracking, and resolution. The system's primary aim is to be reliable and traceable, and this deferment protects that focus. Introducing a third-party notification service such as SMS or email would bring external dependencies, added cost, and integration risk that isn't justified until the core platform is stable. In line with the constraints, this decision protects the project's schedule and cost boundaries and reduces the risk of an unfamiliar third-party integration destabilising features more directly tied to the stated business need.

Functional Requirements

ID	Requirement	Source	Priority	Acceptance Criteria
FR-001	Requester can submit a new service request with relevant details.	Requester in scope	Must	Request will be saved using the ID and status "Submitted" once mandatory

				fields are completed.
FR-002	Requester can categorise their request.	Requester in-scope	Must	Submission will be blocked unless a category is selected.
FR-003	Requester can view the status of their submitted request	Requester in-scope	Must	Status updates visible to Requester in real time as Staff changes it.
FR-004	Requester can view a list of their previous requests	Requester in-scope	Should	"My Requests" list all past submissions with status and date
FR-005	Requester can view, update or delete their submitted requests	Stakeholder Analysis-Staff/Requester conflict	Should	Editing allowed only while status is "Submitted", locked once staff accepts it.
FR-006	Staff can view requests relevant to them and search/filter/sort	Staff in scope	Must	Filters return only matching requests.
FR-007	Staff can view full request details	Staff in scope	Must	All submitted fields visible on one screen
FR-008	Staff can assign or accept responsibility for a request	Staff in scope	Must	Accepting sets status to "In progress" and records the staff member as the owner
Fr-009	Staff can update request status through	Staff in scope	Must	Submitted-> In progress-

	controlled transitions			>Resolved->closed
FR-010	Staff can record actions, comments or resolution notes on a request	Staff in scope	Should	Notes are timestamped
FR-011	Management can view service activity	Management in scope	Must	Counts of open or overdue data
FR-012	Management can view requests by category for basic reporting	Management in scope	Should	Category filter returns only matching requests
FR-013	System enforces role-based access so each role sees only what's relevant to them	Role-based access	Must	Requesters see only their own requests; staff cannot access Management reports

Non-functional Requirements

ID	Requirement	Source	Priority	Acceptance Criteria
NFR-001	Pages should load within an acceptable time	Quality	Must	Submission pages should load in 3 seconds
NFR-002	User data is protected	Security	Must	No data should be accessible without authentication
NFR-003	Works on standard browsers	Quality	Should	Core features function on the latest two versions of Chrome, Edge

Requirements Traceability Matrix

ID	Stakeholder	Requirement Summary	Acceptance Criteria	Status
FR-001	Requester	Submit a new service request	Saved with unique ID, status "Submitted"	Baselined
FR-002	Requester	Categorise the request	Submission blocked without a category	Baselined
FR-003	Requester	View request status	Status visible and updates in real time	Baselined
FR-004	Requester	View request history	"My Requests" lists past submissions	Baselined
FR-005	Requester	Update/withdraw a request	Editable only while "Submitted"	Baselined
FR-006	Staff	Search/filter/sort requests	Filters return matching requests only	Baselined
FR-007	Staff	View full request details	All fields visible on one screen	Baselined
FR-008	Staff	Accept/assign a request	Accepting sets status to "In Progress", records owner	Baselined; traces from the Staff stakeholder need
FR-009	Staff	Update status via controlled transitions	Only Submitted→In Progress→Resolved→Closed allowed	Baselined
FR-010	Staff	Add notes/comments	Notes timestamped and saved to history	Baselined
FR-011	Management	View activity by status	Counts match live data	Baselined
FR-012	Management	View requests by category	Category filter returns matches	Baselined

FR-013	Role-based access	Restrict view by role	Requesters/Staff/Management see only their scope	Baselined
NFR-001	Quality	Acceptable page load time	No more than 3 seconds should be spent loading the dashboard	Baselined
NFR-002	Security	Protect user data	Passwords hashed, no unauthenticated access	Baselined
NFR-003	Quality	Works on standard browsers	Functions on the latest two Chrom or Edge	Baselined

CONSTRAINTS AND TRADE-OFFs ANALYSIS

a) Scope Constraints:

- It is important to note that Milestone 1 is restricted to engineering foundations, and the final technology decision, software architecture, database schemas, UI implementation, APIs, and final production deployment are not included.

b) Schedule & Resource Constraints:

- Under tight milestones and with only 3 team members, it's a real challenge to balance rigorous documentation, risk registries and GitHub governance.

c) Quality & Security Constraints:

- Non-functional requirements (e.g., page load times less than 3 seconds, strict authentication requirements) should be expected at a high level now.

d) Trade-Off / Ripple Effect:

- Project scope is kept to a minimum and automated notifications (SMS/email) are not a part of the project the project's scope and cost are maintained as contained.

Initial Risk Register

b) Risk Item 1 (High Priority):

- Incomplete requirements traceability – causing misaligned acceptance criteria and failing testability in later phases.
- Probability & Impact: Medium probability, High impact.
- Mitigation: Maintain continuous traceability of Requirements Traceability Matrix (RTM) with the needs of the stakeholders.
- Contingency: Do a cross-team audit of RTM prior to baseline sign-off.

c) Risk Item 2:

- Using AI-based tools too heavily which leads to content that is not verified or compliance issues.
- Probability & Impact: Medium probability, medium impact.

- Mitigation: Require human verification of all AI inputs and add those inputted to the AI Usage Register.
- Contingency: Double check all documentations manually and make sure to delete the unverified text before you submit.

2) Forward Engineering Considerations Register

- Register Security & Access Control:
Should be taken into account at this stage to prevent having to add security models after the work was completed.
- Testability:
The requirements should be decomposed into testable criteria of acceptance, including the future, in order to have success with automated and manual test framework.
- Deployment Environment:
Knowing where Civic Connect will be deployed helps reduce friction in the deployment process.
- Data Migration & Recovery:
Ahead of the curve with data volume recognition prevents incorrect database schema decisions.
- Security:
Provides security features for future services to meet the security policies of the system.
- Observability:
Planning system logging and audit trails now ensures transparency of operation during future maintenance. The operational cost of assessing resource footprints avoids expensive cloud or infrastructure architectures going down the pipeline.

Engineering Decision Log

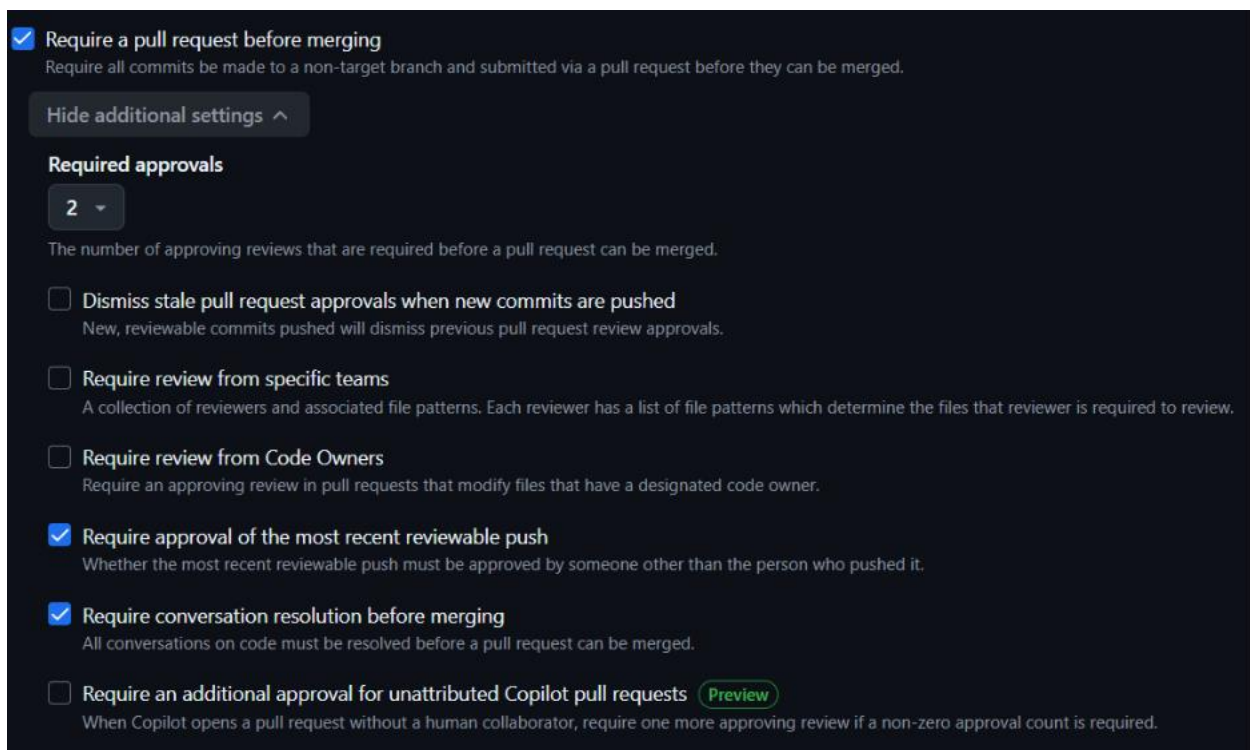
- Decision 1 (Made):
 - Using a protected main branch and a two-reviewer model to ensure version control compliance on GitHub.
 - Justification: Enforces project-wide control and auditability per the Master Project Brief.

- Decision 2 (Deliberately Deferred):
 - Deferral of automated notifications (SMS/email) and advanced management reporting dashboards.
 - Justification: Focus on the core request lifecycle (submission, assignment, tracking, resolution) is at present; adding third-party integrations adds a lot of unnecessary cost and integration risk.

Repository and Team Governance

Github governance

Github governance refers to using rules and controls to manage the development of a software product. For the project, social coding, which is an agile approach for team development, will be used to allow for a collaborative environment (Marius Golombeck, Orłowski and Tübben, 2018). For the governance of the repository, there will be protection rules for the main branch. According to Github (n.d), a custom rule can be applied to a branch, requiring two or more collaborators to approve a request to merge to the main branch. Using this protection rule will help ensure that every piece of code committed to the main branch is tested before approval. It will also ensure that bulk uploads of content that is unverified will not be done.



The screenshot shows the 'Require a pull request before merging' settings for a GitHub repository. The settings are as follows:

- ☒ **Require a pull request before merging**
Require all commits be made to a non-target branch and submitted via a pull request before they can be merged.
- Hide additional settings** ^
- Required approvals**
2 (dropdown menu)
The number of approving reviews that are required before a pull request can be merged.
- ☐ **Dismiss stale pull request approvals when new commits are pushed**
New, reviewable commits pushed will dismiss previous pull request review approvals.
- ☐ **Require review from specific teams**
A collection of reviewers and associated file patterns. Each reviewer has a list of file patterns which determine the files that reviewer is required to review.
- ☐ **Require review from Code Owners**
Require an approving review in pull requests that modify files that have a designated code owner.
- ☒ **Require approval of the most recent reviewable push**
Whether the most recent reviewable push must be approved by someone other than the person who pushed it.
- ☒ **Require conversation resolution before merging**
All conversations on code must be resolved before a pull request can be merged.
- ☐ **Require an additional approval for unattributed Copilot pull requests** Preview
When Copilot opens a pull request without a human collaborator, require one more approving review if a non-zero approval count is required.

Above is the rule set that will be applied for the other milestones of our project.

Team Governance

The following will be done to ensure that at the end, a functional software product that meets the needs and requirements of the stakeholders:

- Using a protected main branch in the repository

This will ensure that teams can work on their parts with full autonomy on separate branches and only commit work to the main branch after review.

- Two collaborators out of the three will have to approve any changes that will have to be merged to our main branch.
- Pull requests with detailed descriptions, branches, and commits will be used throughout the project.

AI Compliance and controls

AI compliance refers to ensuring that AI used adheres to the legal and ethical standards (McGrath and Jonker, 2024). It is also about making sure that any decision taken with the assistance of AI is well-informed and it goes through human review to maintain transparency and trust between the team and the stakeholders (McGrath and Jonker, 2024). For the AI compliance part of our project, we will focus on the following:

- **Planning**

Before using any AI assisted coding tools, the focus will be on planning the architecture, testing plans, and integration points of the project first (Sreejith Sreejayan, 2025). Then a plan on how to handle AI generated output will be handled.

- **Considering constraints during development**

Instead of just sending prompts to AI models, during development, constraints will be sent to the AI models to assist in building individual modules to prevent multiple errors caused by prompting AI to code everything (Sreejith Sreejayan, 2025).

- **Prioritizing the understanding of the functionality of the code**

The team will focus on understanding the implementation of the code, the data flow in the system, and how to debug code when there might be a requirement change (Sreejith Sreejayan, 2025).

- **Security focused AI assisted development**

All AI code must be treated as potentially malicious as up to 35.8% of AI generated code can cause security weaknesses in software systems (Sreejith Sreejayan, 2025). As a precaution the following rules will be followed when developing the software system:

- API keys or any sensitive information should not be added into any AI prompts.
- Placeholder values should be used in prompts instead of using real values.
- AI generated code should not be deployed without human review.

- **AI usage register**

Throughout the project, we will be using an AI usage register to document all the AI outputs that will be used in the project. This will include a unique ID for each output, its source, purpose, owner, and risk.

CONCLUSION

By enforcing the following branch protection rules, review approval rules, and structure for pull requests and commits, accountability and transparency throughout the project will be maintained. These rules along with AI compliance will ensure that AI tools will be used to further improve the project and to prevent any defects in the software project. These will ensure that the integrity of the software project in this milestone through to the last milestone.

References

ServiceNow, 2026. *ServiceNow*. [Online] Available at:

<https://www.servicenow.com/products/itsm/what-is-service-request-management.html>

[Accessed 8 September 2026].

Belgium Campus ITversity, 2026. *SEN381 Software Engineering 381: CivicConnect Master Project Brief*. Pretoria: Belgium Campus ITversity.

Belgium Campus ITversity, 2026. *SEN381 Project 1: Requirements, Traceability, and Scope*. Pretoria: Belgium Campus ITversity.

Somerville, I., 2015. *Software Engineering*. 10th ed. Boston: Pearson.

Pressman, R.S. & Maxim, B.R., 2020. *Software Engineering: A Practitioner's Approach*. 9th ed. New York: McGraw-Hill Education.

ServiceNow, 2026. *ServiceNow*. [Online]
Available at: <https://www.servicenow.com/products/itsm/what-is-service-request-management.html>
[Accessed 8 september 2026].

Atlassian (2024). *What is version control*. [online] Atlassian. Available at:
<https://www.atlassian.com/git/tutorials/what-is-version-control> [Accessed 8 Sept. 2026].

David Rendon (2026). *How GitHub enterprise governance lets you ship secure code at scale without slowing down your....* [online] Medium. Available at:
<https://medium.com/codex/how-github-enterprise-governance-lets-you-ship-secure-code-at-scale-without-slowing-down-your-90b83a0dba2f> [Accessed 8 Sept. 2026].

Github (n.d.). *About protected branches*. [online] GitHub Docs. Available at:
<https://docs.github.com/en/repositories/configuring-branches-and-merges-in-your-repository/managing-protected-branches/about-protected-branches> [Accessed 8 Sept. 2026].

Marius Golombeck, Orlowski, H. and Tübben, S. (2018). *Collaborative Software Development with GitHub® Projects*. [online] Available at:

https://www.researchgate.net/publication/330162832_Collaborative_Software_Development_with_GitHubR_Projects [Accessed 8 Sept. 2026].

McGrath, A. and Jonker, A. (2024). *AI compliance*. [online] Ibm.com. Available at: <https://www.ibm.com/think/insights/ai-compliance> [Accessed 9 Sept. 2026].

Sreejith Sreejayan (2025). *How to use AI for your software project: A complete guide for 2025*. [online] Medium. Available at: <https://medium.com/@the-sreejith/how-to-use-ai-for-your-software-project-a-complete-guide-for-2025-3dda56c3fb23> [Accessed 9 Sept. 2026].